

作业4

1.

```
int readCount = 0;
int writeCount = 0;
semaphore read = 1;
semaphore readCountSignal = 1;
semaphore writeCountSignal = 1;
semaphore fileSrc = 1;

reader()
{
    while(true)
    {
        P(read);          //申请令牌
        P(readCountSignal); //申请读者队列计数器

        if(!readCount)     //如果读者队列为空，申请文件资源
            P(fileSrc);
        readCount++;

        V(readCountSignal); //释放读者计数器资源
        V(read);           //释放令牌

        read;

        P(readCountSignal); //申请读者计数器资源
        readCount--;
        if(!readCount)     //如果读者队列为空，释放文件资源
            V(fileSrc);
        V(readCountSignal); //释放读者计数器资源
    }
}

writer()
{
    while(true)
    {
        P(writeCountSignal); //申请写者计数器资源
        if(!writeCount)     //如果写者队列为空则申请令牌
            P(read);
        writeCount++;
        V(writeCountSignal); //释放写者计数器资源
        P(fileSrc);         //申请文件资源

        write;

        V(fileSrc);        //释放文件资源
        P(writeCountSignal); //申请写者计数器资源
        writeCount--;
        if(!writeCount)    //如果写者队列为空则释放令牌
            V(read);
    }
}
```

```

        V(writeCountSignal); //释放写者计数器资源
    }

}

```

2.

```

int eating = 0, waiting = 0;
Semaphore mutex = 1, queue = 1;
bool must_wait = false;

Customer(){
    P(mutex);
    if (must_wait){
        waiting++;
        V(mutex);
        P(queue);
    }
    else {
        eating++;
        must_wait = (eating == 5)

        V(mutex);
    }

    Eat();
    P(mutex);
    eating--;
    if (eating == 0){
        int n = min(5, waiting);
        waiting -= n;
        eating +=n;
        must_wait = (eating == 5)
        for(int i = 0; i<n; i++)
            V(queue);
    }
    V(mutex);
}

```

3.

```

Semaphore empty = N; // 假设初始条件下缓冲区有N个空位
Semaphore mutex = 1;
Semaphore odd = 0; // 缓冲区中奇数数量
Semaphore even = 0; //偶数数量

void P1(){
    int integer;
    while(true){
        integer = produce();
        P(empty);
        P(mutex);
        put();
        V(mutex);
        if(integer %2 == 0){
            V(even);

```

```

    } else {
        V(odd);
    }
}
}

void P2(){
    while(true){
        P(odd);
        P(mutex);
        getodd();
        V(mutex);
        V(empty);
        countodd();
    }
}

void P3(){
    while(true){
        P(even);
        P(mutex);
        geteven();
        V(mutex);
        V(empty);
        counteven();
    }
}
}

```

4.

Semaphore insertMutex = 1, searchMutex = 1; // 保护int变量
 Semaphore No_search = 1; // 顾名思义，为1时没有搜索进程访问
 Semaphore No_insert = 1; // 为1时没有插入进程访问
 //当上述两个信号量同时为1，删除者才可以进行删除操作

```

int searcher = 0, inserter = 0;
void Search(){
    P(searchMutex);
    searcher++;
    if (searcher == 1) // 第一个进来的搜索者加锁
        P(No_search);
    V(searchMutex);
    Searching(); // 访问临界区，多个搜索无需互斥
    P(searchMutex);
    searcher--;
    if (searcher == 0)
        V(No_search); // 表示此时没有搜索线程在进行，解锁
    V(searchMutex);
}

void Insert(){
    P(insertMutex);
    inserter++;
    if (inserter == 1)
        P(No_insert);
    V(insertMutex);
}

```

```
P(insertMutex); // 既然可以和搜索线程并行，那么不用管Searcher
    Inserting(); // 访问临界区，多个插入者要互斥访问，一次一个insert
V(insertMutex);

P(insertMutex);
    inserter--;
    if (inserter == 0)
        V(No_insert); // 解锁，可唤醒删除者
V(insertMutex);
}

void Delete() { // 删除线程与其他任何线程互斥
    P(No_search);
    P(No_insert); // 若为1则可进入，这个信号量顺便也可以当作删除者的互斥保护
    Deleting(); // 搜索和插入线程都没，成功进入临界区
    V(No_insert);
    V(No_search);
}
```